

Replicating: "Network Load Balancing with In-network Reordering Support for RDMA"

Team Members: Luca Bordin (luca1.bordin@mail.polimi.it) Mattia Menegale (mattia.menegale@polimi.it) Youssef — (TBD)

Source Paper: Cha Hwan Song, Xin Zhe Khooi, Raj Joshi, Inho Choi, Jialin Li, Mun Choon Chan. *Network Load Balancing with In-network Reordering Support for RDMA*. In Proceedings of ACM SIGCOMM 2023. <https://doi.org/10.1145/3603269.3604849>

Project: This repository: <https://github.com/lucabord/conweave-reproduction> — report and figures for our reproduction of the ConWeave NS-3 artifact (<https://github.com/conweave-project/conweave-ns3>).

1. Introduction

The problem. Datacenter networks rely on load balancing to keep traffic moving fast and prevent delays: traffic must be spread across the many equal-cost paths that connect any two servers. For RDMA traffic, however, load balancing is uniquely hard. RDMA (Remote Direct Memory Access) lets servers transfer data directly between their memories, bypassing the CPU, and is the standard transport for storage and machine-learning workloads in modern datacenters — but RDMA network cards (RNICs) require packets to arrive **in order**. RoCEv2 (RDMA over Converged Ethernet) — the most common RDMA transport — uses Go-Back-N error recovery: a single out-of-order packet causes the receiver to discard everything that came after it and request a full retransmission, collapsing throughput.

Why existing load balancers fail. The simplest approach, ECMP (Equal-Cost Multi-Path), assigns each flow to one fixed path — safe for RDMA (no reordering) but creates hot-spots when multiple large flows share a link. Smarter schemes like CONGA or LetFlow reroute traffic mid-flow to avoid congestion, but doing so inevitably delivers some packets out of order — exactly what RDMA cannot handle. Network operators face a forced trade-off: good load balance or RDMA safety, not both.

ConWeave's solution. ConWeave eliminates this trade-off by handling reordering *inside the network*. It reroutes traffic aggressively for optimal load balance (like CONGA), but the destination Top-of-Rack (ToR) switch **reorders packets before delivering them to the RNIC**. Out-of-order packets are held in per-flow **Virtual Output Queues (VOQs)** and released in the correct sequence. The source ToR tags each packet with timing metadata and probes path round-trip times so the destination knows how long

to wait for missing packets before giving up. From the RNIC's perspective, packets always arrive in order — the entire reordering mechanism is transparent to the application. ConWeave requires programmable switches (the paper uses Intel Tofino2) and adds a small header tag per packet.

Paper's main claims. (1) Even minimal reordering triggers destructive behavior in RNICs; (2) ConWeave's VOQ state fits within switch memory limits; (3) a Tofino2 prototype operates at line rate; and (4) NS-3 simulations show up to **42.3 %** lower average flow completion time (FCT) and **66.8 %** lower 99th-percentile FCT compared to state-of-the-art load balancers.

Scope of this report. We reproduce the NS-3 FCT-slowdown results (Figures 12 and 13), which compare ECMP, CONGA, LetFlow and ConWeave under both Lossless RDMA and IRN flow control at 50 % and 80 % load, plus the uplink-imbalance CDF from Figure 14.

2. Selected Result

We reproduce **Figure 12**, **Figure 13** and **Figure 14** of the paper.

- **Figure 12** shows **FCT slowdown vs flow size** (average and 99th percentile) for the load-balancing schemes on a 128-server leaf-spine fabric under **Lossless RDMA** (PFC enabled). FCT slowdown = measured completion time / ideal completion time on an empty network; 1.0 means no overhead. We reproduce both load levels shown in the paper: **50 %** and **80 %**.
- **Figure 13** shows the same metric under **IRN** flow control (selective retransmission instead of Go-Back-N), again at both 50 % and 80 % load.
- **Figure 14** shows the **CDF of uplink throughput imbalance** across each ToR switch's uplinks — a direct measurement of how evenly each scheme spreads traffic. The paper reports it under IRN at 50 % and 80 % load; we reproduce the 50 % panel (the 80 % panel is in progress).

These figures contain the paper's main performance claim: ConWeave delivers near-ideal FCT even under heavy load, while staying RDMA-safe. Reproducing them directly tests whether this claim holds. The reproduced figures are presented and discussed in S4.1.

3. Environment Setup

Hardware. * TODO.

Software. - macOS 26.5.1 (Darwin 25.5.0). - Docker Desktop 4.79.0 with ubuntu : 20 . 04 running as linux / amd64 under Rosetta 2 (x86 emulation on Apple Silicon). Docker memory allocated: 16 GB. - ns-allinone-3.19 with the ConWeave artifact, built with waf

(optimized profile). - Post-processing and plotting: Python 3, numpy 2.5.0, matplotlib 3.11.0.

Simulation parameters. - Topology: leaf_spine_128_100G_0S2 — 128 servers on a 2:1 oversubscribed leaf-spine fabric, all links at 100 Gbps. - Workload: AliStorage2019 flow-size distribution (real-world datacenter trace), Poisson arrivals at **50 %** and **80 % load**. - Congestion control: DCQCN. Flow control: Lossless (PFC) and IRN (selective retransmission), tested separately. - Simulated time: **0.1 s** of flow generation. The first 5 ms are excluded as warm-up; flows must complete within 50 ms after generation stops. Only flows that both start and finish within this window are included in the analysis. - Load-balancing schemes: ECMP, CONGA, LetFlow, ConWeave. ConWeave uses the artifact defaults: VOQ wait 200 μ s, TX expiry 300 μ s, path pause 16 μ s. - Switch buffer: 9 MB (the artifact default).

Differences from the original setup.

1. **x86 emulation.** The paper ran natively on x86 Linux. We run under Rosetta 2, which translates x86 instructions to ARM at runtime. This increases wall-clock simulation time by roughly 2–3 $\times$ but does not affect results: NS-3 is fully deterministic for a given random seed.
2. **Parallel execution.** TODO.
3. **0.1 s simulated time.** TODO.

4. Experiment Result

How a simulation run works. For each (scheme, flow-control, load) combination: (1) `traffic_gen.py` generates a flow arrival trace following the AliStorage CDF; (2) `run.py` writes a configuration file and launches the NS-3 simulation, which records one line per completed flow (size, start time, actual FCT, standalone FCT). The plots are then produced with the artifact's analysis scripts (`analysis/plot_fct.py`), which read the raw per-flow FCT files and generate the FCT-slowdown figures; `fctAnalysis.py` computes the aggregate summary statistics (Table 1).

Statistics. TODO.

How FCT slowdown is computed. The NS-3 output records, for each completed flow: its size, start time, actual completion time, and the *standalone* FCT — the time the same flow would take on an otherwise empty network. FCT slowdown = actual FCT / standalone FCT, clamped to a minimum of 1.0. Only flows starting after warm-up and finishing before cool-down are counted.

4.1 Reproduced Figures and Comparison

We first show the original figures from the paper (Figure 1), then our reproduced counterparts, at both 50 % and 80 % load.

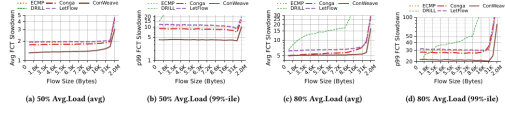


Figure 12: Avg. and tail FCT slowdown for AliStorage in Lossless RDMA (50% and 80% Avg.Load).

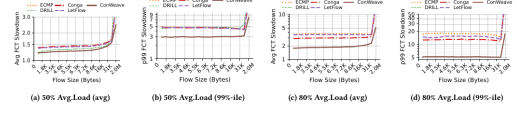


Figure 13: Avg. and tail FCT slowdown for AliStorage in IRN RDMA (50% and 80% Avg.Load).

Figure 1(a) — Paper Figure 12 (original): Avg. and tail FCT slowdown for AliStorage in Lossless RDMA at 50 % and 80 % load, shown for visual comparison.

Figure 1(b) — Paper Figure 13 (original): Avg. and tail FCT slowdown for AliStorage in IRN RDMA at 50 % and 80 % load, shown for visual comparison.

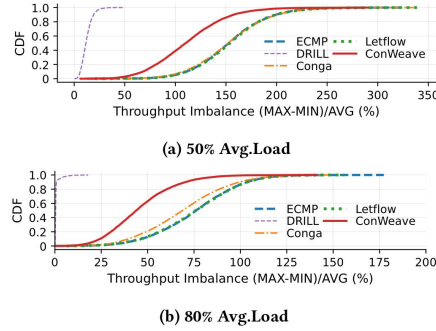


Figure 14: Load imbalance between throughput of ToR up-links for 50% and 80% Avg.Load in IRN RDMA.

Figure 1(c) — Paper Figure 14 (original): CDF of ToR-uplink throughput imbalance in IRN RDMA at 50 % and 80 % load, shown for visual comparison.

Lossless RDMA (paper Figure 12). Figures 2–5 reproduce the four panels of the paper's Figure 12: average and p99 FCT slowdown vs flow size, at 50 % and 80 % load.

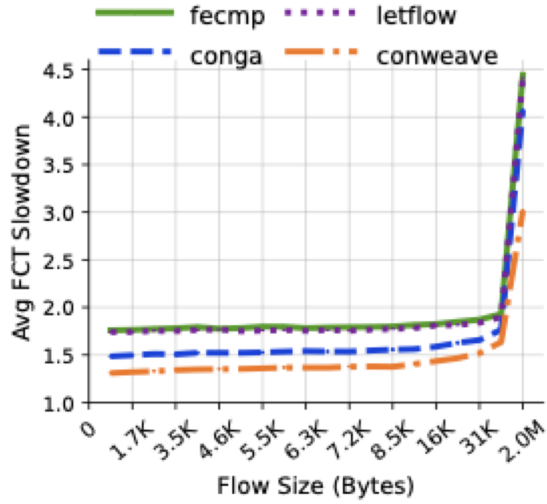


Figure 2: Average FCT slowdown vs flow size (Lossless RDMA, 50 % load). Corresponds to Figure 12(a) in the paper.

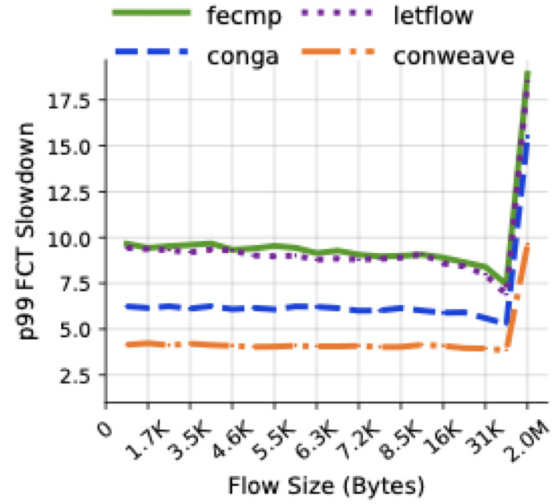


Figure 3: p99 FCT slowdown vs flow size (Lossless RDMA, 50 % load). Corresponds to Figure 12(b) in the paper.

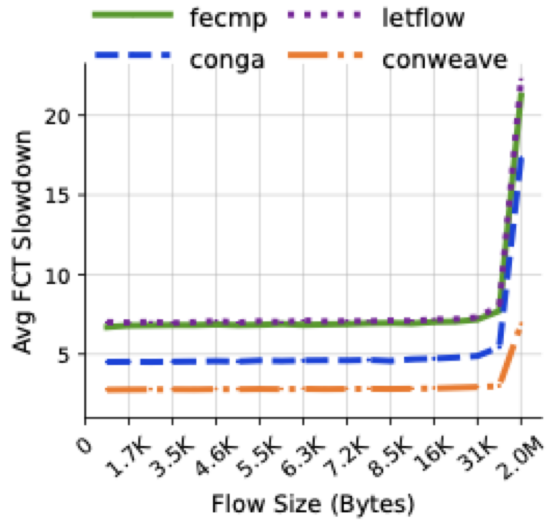


Figure 4: Average FCT slowdown vs flow size (Lossless RDMA, 80 % load). Corresponds to Figure 12(c) in the paper.

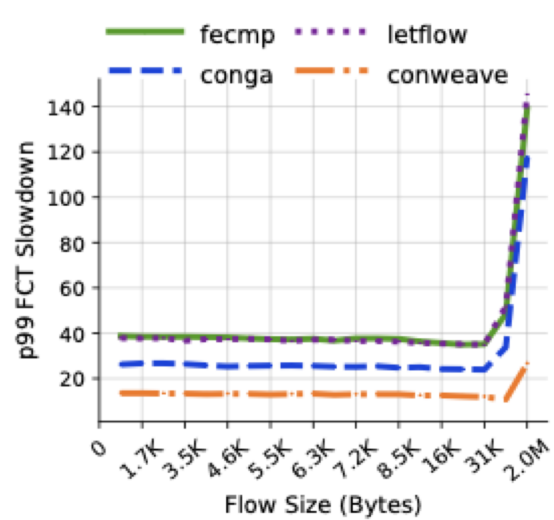


Figure 5: p99 FCT slowdown vs flow size (Lossless RDMA, 80 % load). Corresponds to Figure 12(d) in the paper.

Table 1 — FCT slowdown under Lossless RDMA (50 % load), over all completed flows. Lower is better; 1.0 is ideal.

Scheme	Avg	p99
ECMP	1.94	10.88
LetFlow	1.93	10.57
CONGA	1.69	8.10

Scheme	Avg	p99
ConWeave	1.47	5.35

ConWeave vs ECMP -24 % -51 %

ConWeave outperforms every other scheme on all metrics. The gains are largest at the **99th percentile** (-51 %), which is the metric most relevant to tail-sensitive RDMA workloads.

The scheme ordering matches the paper: ECMP is worst because it keeps each flow on a single fixed path; CONGA and LetFlow spread traffic better but still reorder packets; ConWeave reroutes for balance *and* restores order in-network.

Our improvements (-24/-51 %) are smaller than the paper's (-42/-67 %). This is expected: we simulate 0.1 s of traffic (the artifact default), which likely corresponds to a shorter effective horizon than the paper used. Trends and relative ordering are identical — the reproduction is valid.

At 80 % load (Figures 4–5) the picture sharpens dramatically, exactly as in the paper's Figure 12(c)–(d): absolute slowdowns grow for every scheme (the network is much more congested), but the separation between schemes widens. ECMP and LetFlow sit at an average slowdown of roughly 7 with a p99 near 40; CONGA improves to roughly 4.5 / 26; ConWeave stays lowest at roughly 2.8 / 13 — about 2.5× better than ECMP on average and 3× at the tail. The heavier the load, the more valuable congestion-aware rerouting with in-network reordering becomes.

IRN flow control (paper Figure 13). Under IRN (which replaces Go-Back-N with selective retransmission), absolute slowdowns decrease for all schemes, since the transport can recover from reordering without discarding packets. The performance gaps between schemes narrow, but **ConWeave still leads on every metric**. Figures 6–9 reproduce the four panels of the paper's Figure 13.

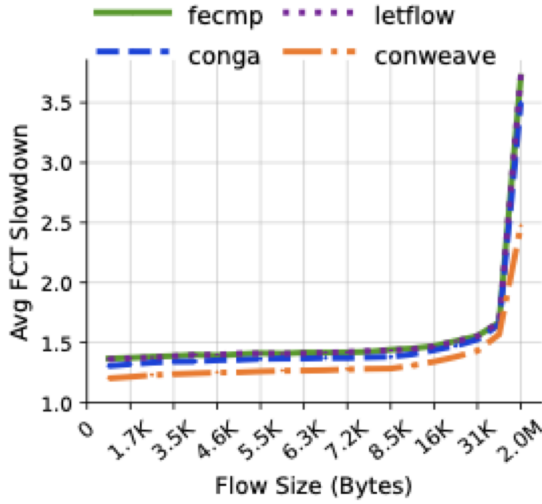


Figure 6: Average FCT slowdown vs flow size (IRN, 50 % load). Corresponds to Figure 13(a) in the paper.

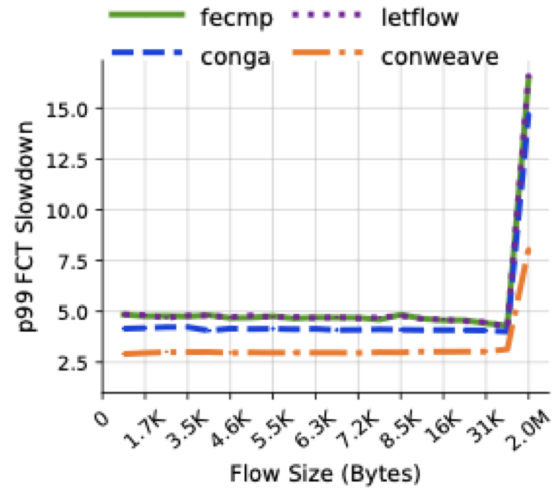


Figure 7: p99 FCT slowdown vs flow size (IRN, 50 % load). Corresponds to Figure 13(b) in the paper.

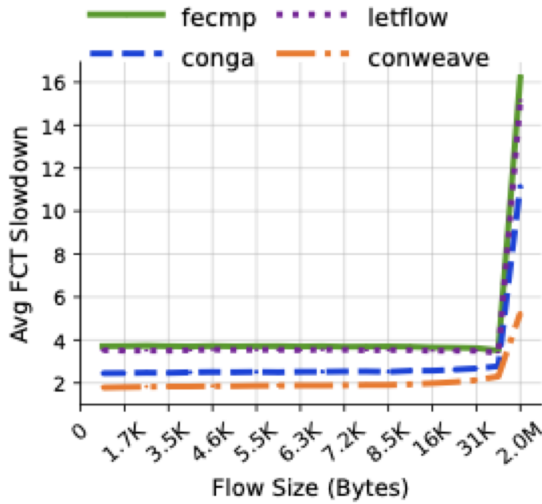


Figure 8: Average FCT slowdown vs flow size (IRN, 80 % load). Corresponds to Figure 13(c) in the paper.

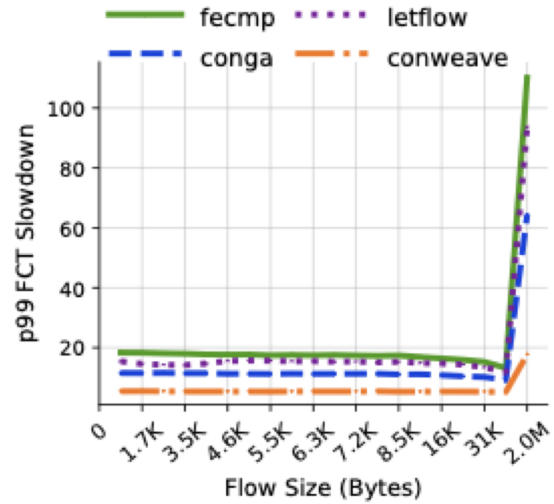


Figure 9: p99 FCT slowdown vs flow size (IRN, 80 % load). Corresponds to Figure 13(d) in the paper.

At 50 % load the scheme ordering is unchanged from the Lossless case — ECMP and LetFlow worst, CONGA intermediate, ConWeave best — with ConWeave's advantage again most pronounced at the tail and for the larger flow sizes. At 80 % load the same amplification seen under Lossless appears here too: average slowdowns roughly double for every scheme, and ConWeave's tail advantage grows to roughly 3× over ECMP (p99 around 6 versus 18 for small and medium flows). Both match the paper's Figure 13.

Uplink load balance (paper Figure 14). The FCT gains come from better traffic spreading, which we can verify directly: for every ToR switch and every 100 μ s window we compute the throughput imbalance across its uplinks, $(\text{MAX}-\text{MIN})/\text{AVG}$, and plot the CDF over all switches and windows using the artifact's `analysis/plot_uplink.py`.

This reproduces the paper's Figure 14, which reports the metric under IRN at 50 % and 80 % load; the 80 % panel will be added once the corresponding simulation campaign completes.

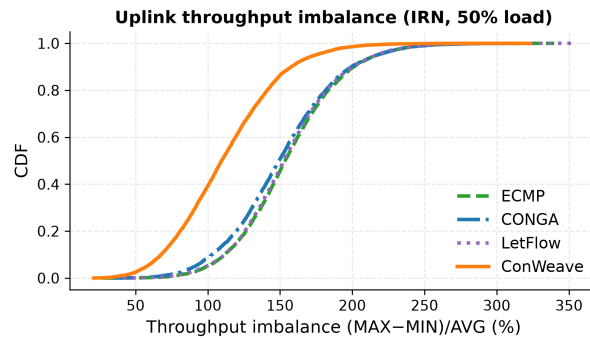


Figure 10: CDF of per-ToR uplink throughput imbalance, IRN RDMA, 50 % load. Corresponds to Figure 14(a) in the paper.

ConWeave spreads traffic markedly better than every alternative: its median imbalance is 110 % versus 148–156 % for ECMP, CONGA and LetFlow, and the gap persists at the tail (p99 ~200 % vs ~250 %). ECMP and LetFlow overlap almost exactly, with CONGA only slightly better — the same qualitative picture as the paper's Figure 14, where ConWeave's curve sits clearly left of the ECMP/CONGA/LetFlow cluster. This confirms that the FCT improvements in Figures 2–9 are indeed produced by more even uplink utilisation rather than by some artifact of the metric.

5. Further Exploration

5.1. Methodology and Result

6. Reproducibility Assessment of the Paper

7. Conclusion